



PORTFOLIO CASE STUDY

AIDA - Dental Family

FULL-STACK PRODUCT CASE STUDY

Building a HIPAA-aware dental clinic system end to end

A production web application that unifies patient records, interactive odontograms, scheduling, role-based staff access, and clinic finances — designed, built, hosted, and operated by one full-stack engineer.

Next.js 16

React 19

TypeScript

InsForge / Postgres

Vercel

Self-hosted Proxmox

AUTHOR

Omar Boza — Full-Stack Developer

Email: omarboza@gmail.com

CV: amazon.github.io/resume

PRODUCT

AIDA — Dental Information & Administration Assistant

Live clinic admin panel for Dental Family.

Screenshots use anonymized / synthetic patient labels.

From fragmented clinic workflows to one operational system

Dental clinics often juggle appointments in one tool, patient notes in another, and payments in spreadsheets. That fragmentation creates missed context, inconsistent billing, and risky access patterns when assistants need scheduling data without full clinical visibility.

AIDA was built as a single administrative panel for Dental Family: clinicians manage charts and odontograms; assistants manage the calendar with minimum-necessary patient data; finances stay linked to clinical visits.

My role

- Product definition & UX
- Frontend architecture
- Backend schema, RLS & edge functions
- Self-hosted Proxmox operations
- Vercel CI/CD & release ownership

Clinical continuity

Patient profiles, medical timeline, attachments, and FDI odontogram history in one place.

Operational scheduling

Week/month calendars, treatment types, doctor assignment, blocked days, notifications.

Clinic economics

Income/expense tracking, pending payments, USD/NIO conversion, KPI dashboards.

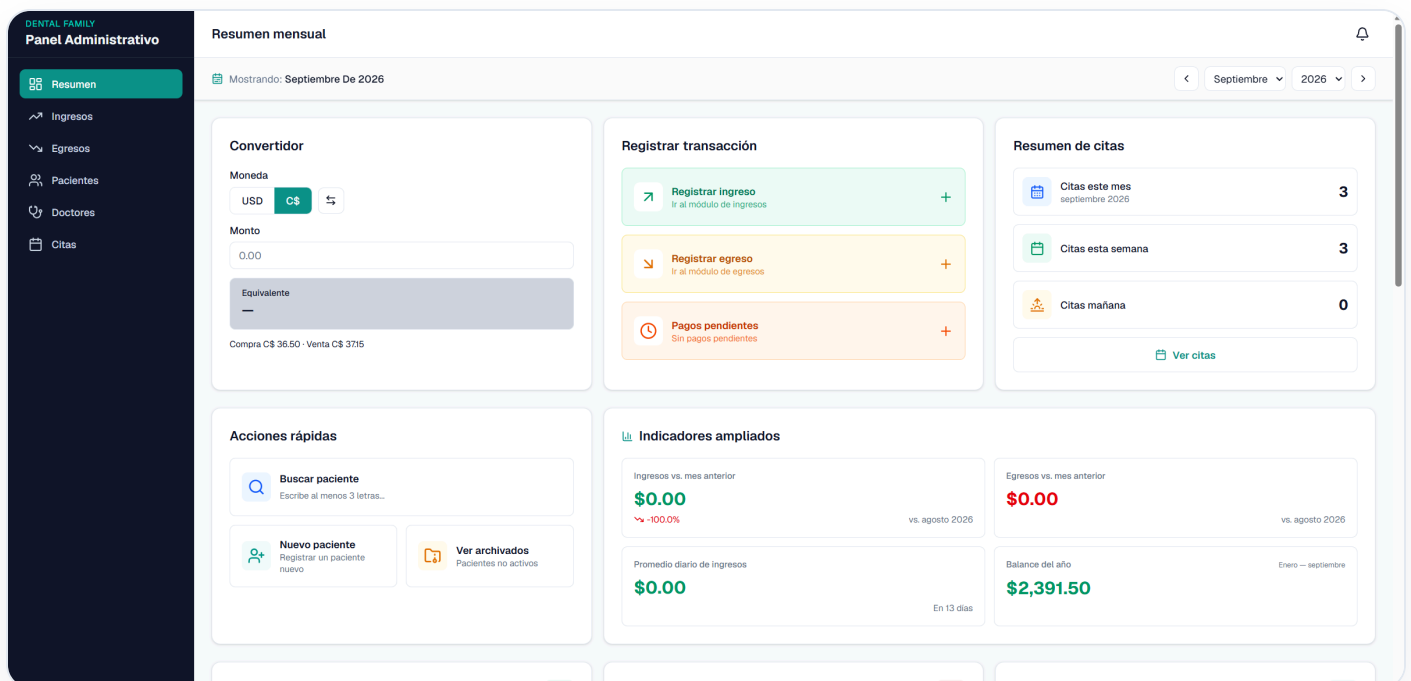


Fig. 1 — Monthly operations dashboard with KPIs, quick actions, and appointment summary.

Patient records that connect care and billing

Each patient profile combines demographics, emergency contact, medical history (rich text), appointment context, clinical timeline entries, linked payments, and media attachments. Creating a clinical note can optionally generate a synchronized income transaction — so the chart and the ledger stay aligned.

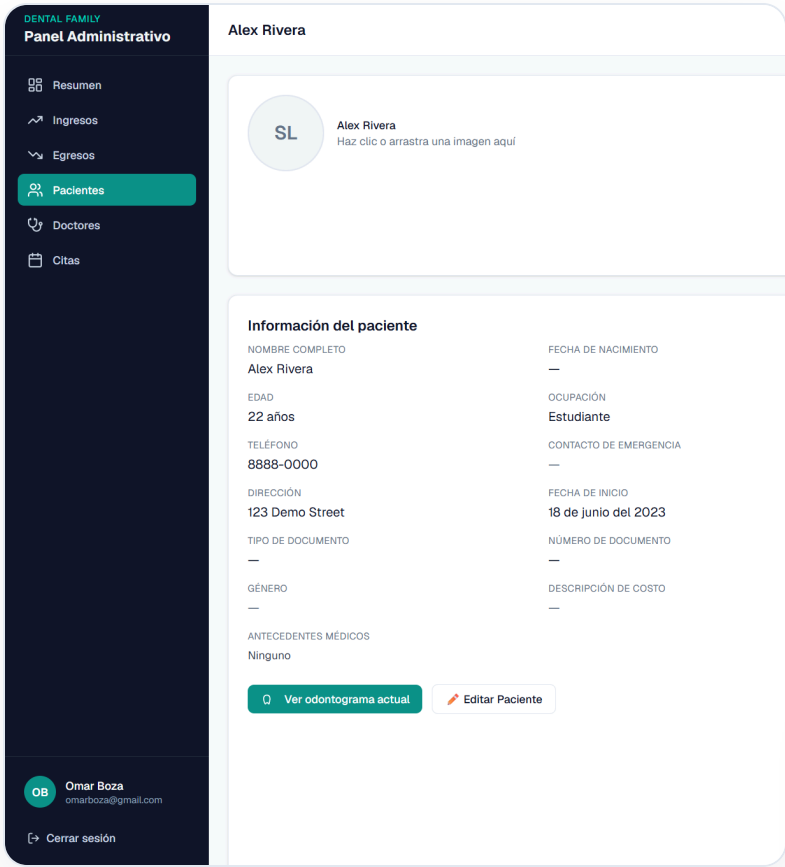


Fig. 2 — Patient detail with next appointment, demographics, and new record composer. Names/phones anonymized for this document.

Highlights

- Draft autosave while editing patient data
- Archive / restore lifecycle with permanent delete cascade
- Medical record timeline with attachments
- Pending vs realized payments per patient
- Photo upload and media gallery

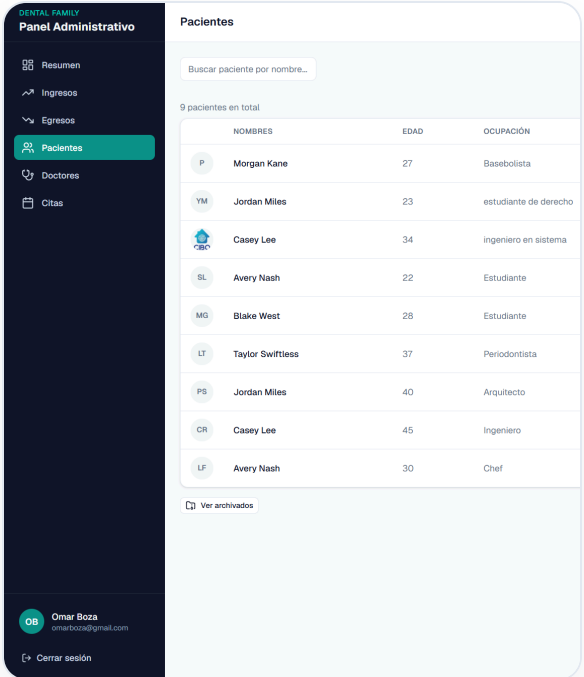


Fig. 3 — Searchable patient directory with role-aware actions.

Interactive FDI odontogram as a first-class chart

The odontogram is not a static image — it is an interactive SVG chart using FDI numbering, multi-anomaly states per tooth (caries, filling, crown, endodontics, implant, and more), and history that travels with each medical record. Coverage analytics on the dashboard show how many active patients have charted odontograms.

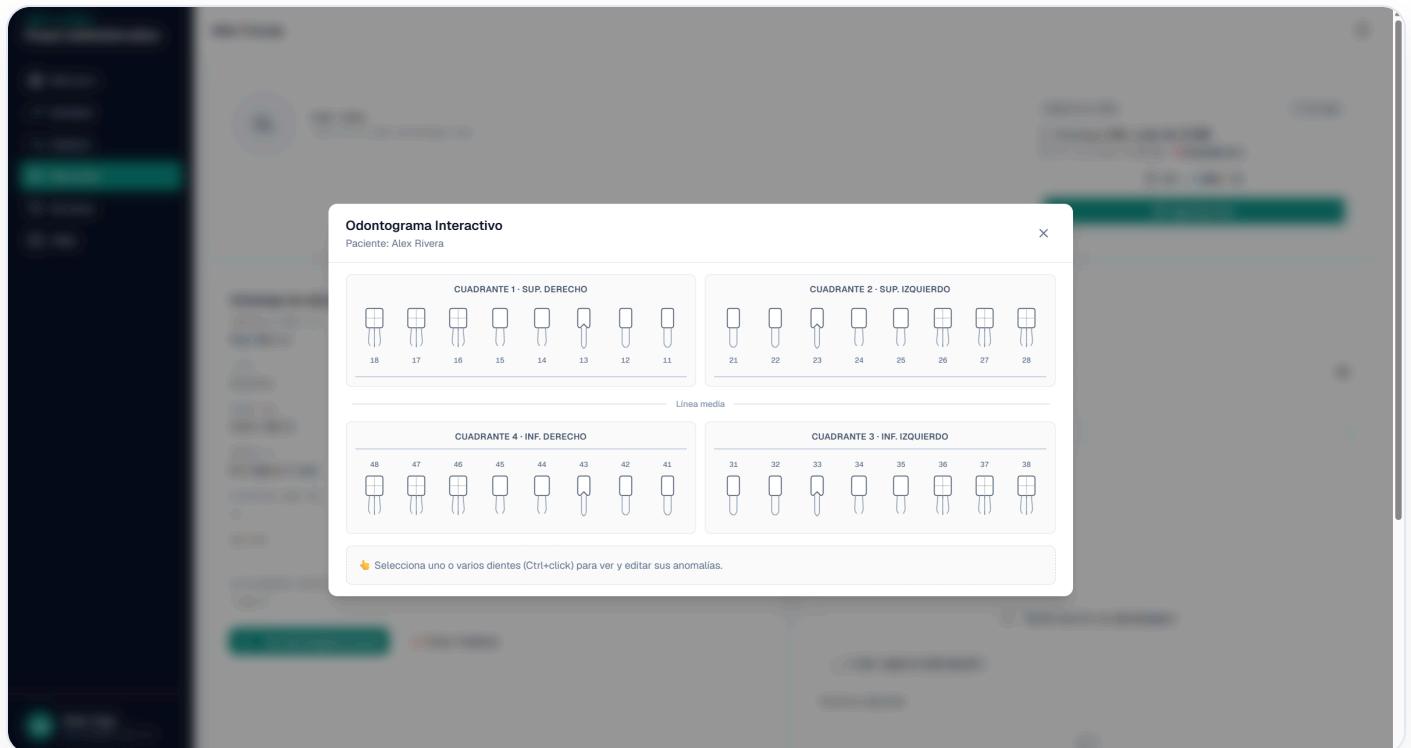


Fig. 4 — Interactive odontogram modal with four-quadrant FDI layout and anomaly editing.

Technical note: Tooth state rules enforce terminal exclusivity (e.g. extracted/absent), combination limits, and FIFO replacement when the anomaly ceiling is reached — keeping chart data clinically consistent instead of free-form tags.

Calendar operations with least-privilege staff access

The appointments module supports monthly overview and weekly time-grid views, treatment-colored events, status tracking, doctor assignment, and unavailable days. Assistants can manage the schedule for the owning clinician while receiving a minimized patient dataset — contact fields for booking, not full clinical/financial records.

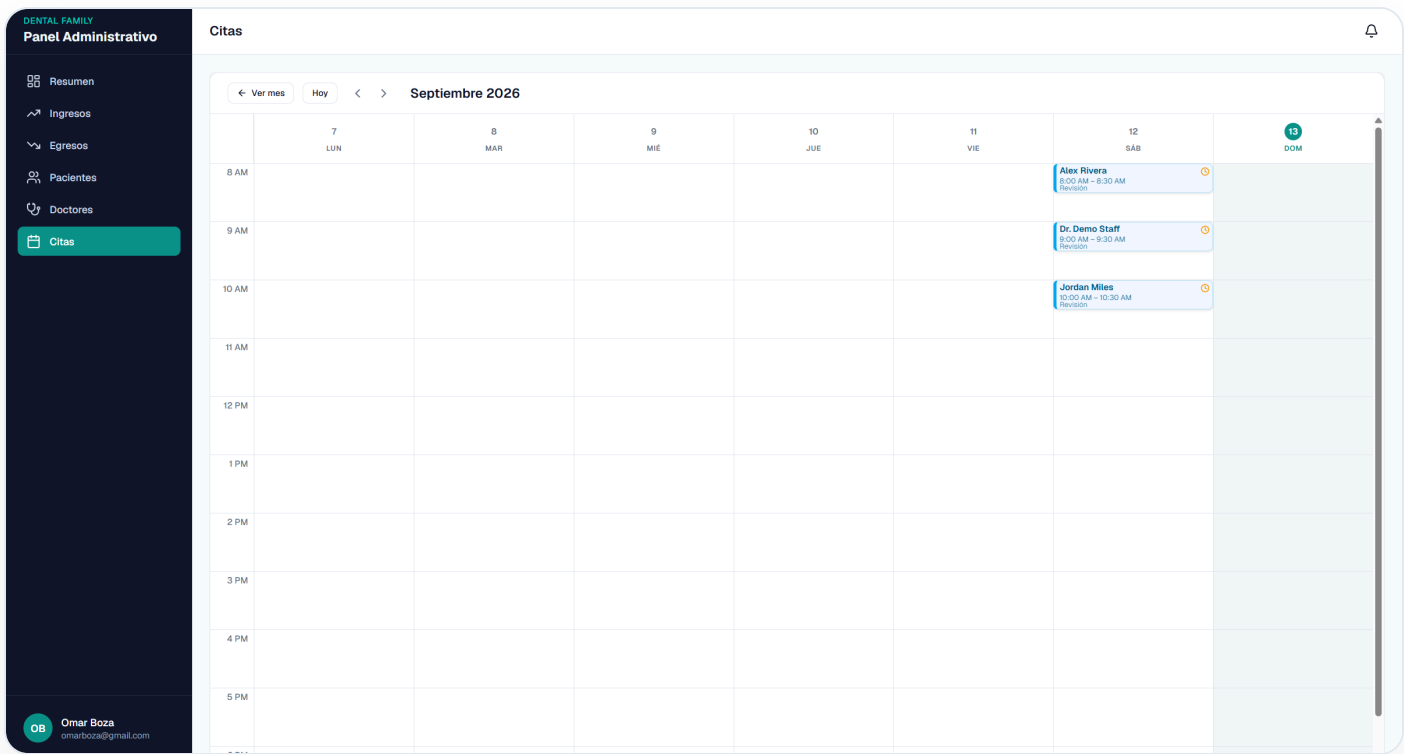


Fig. 5 — Weekly calendar with treatment sessions (patient labels anonymized).

Doctor / Admin

Full access to records, finances, doctors roster, analytics, and clinic settings.

Assistant

Calendar-first UI, limited patient fields via *patients_minimal*, DB triggers restricting sensitive edits.

Route guards

Client RoleGuard + Postgres RLS keyed to `auth.uid()` and `is_assistant_of()`.

Clinic money flows tied to real visits

Income and expense modules support categories, pending payments, patient/entity linking, and dual-currency entry (amounts stored in USD; NIO shown with buy/sell spread). The summary page surfaces month KPIs, year balance, category breakdowns, odontogram coverage, birthday reminders, and recent movements via Recharts.

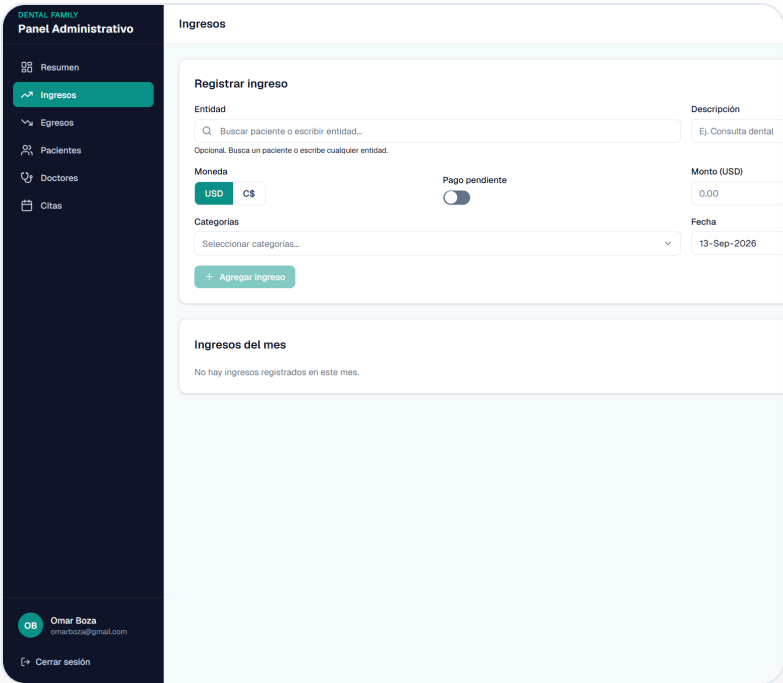


Fig. 6 — Income registration with patient/entity search, categories, and pending toggle.

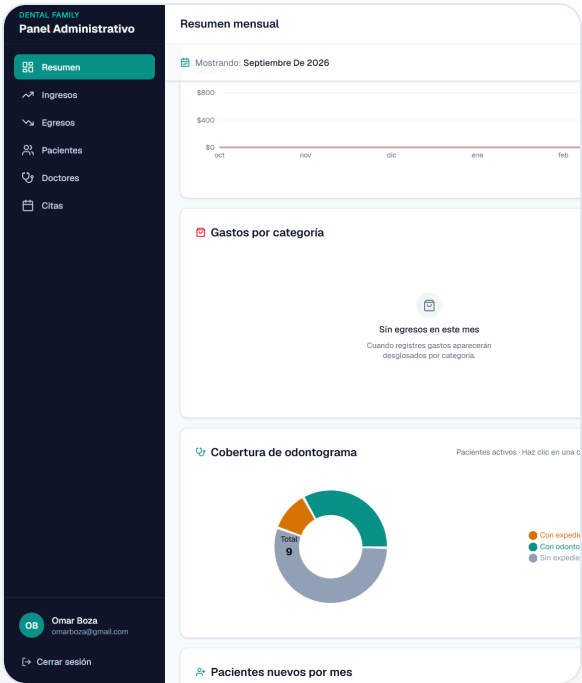


Fig. 7 — Evolution chart, odontogram coverage, and upcoming birthday insights.

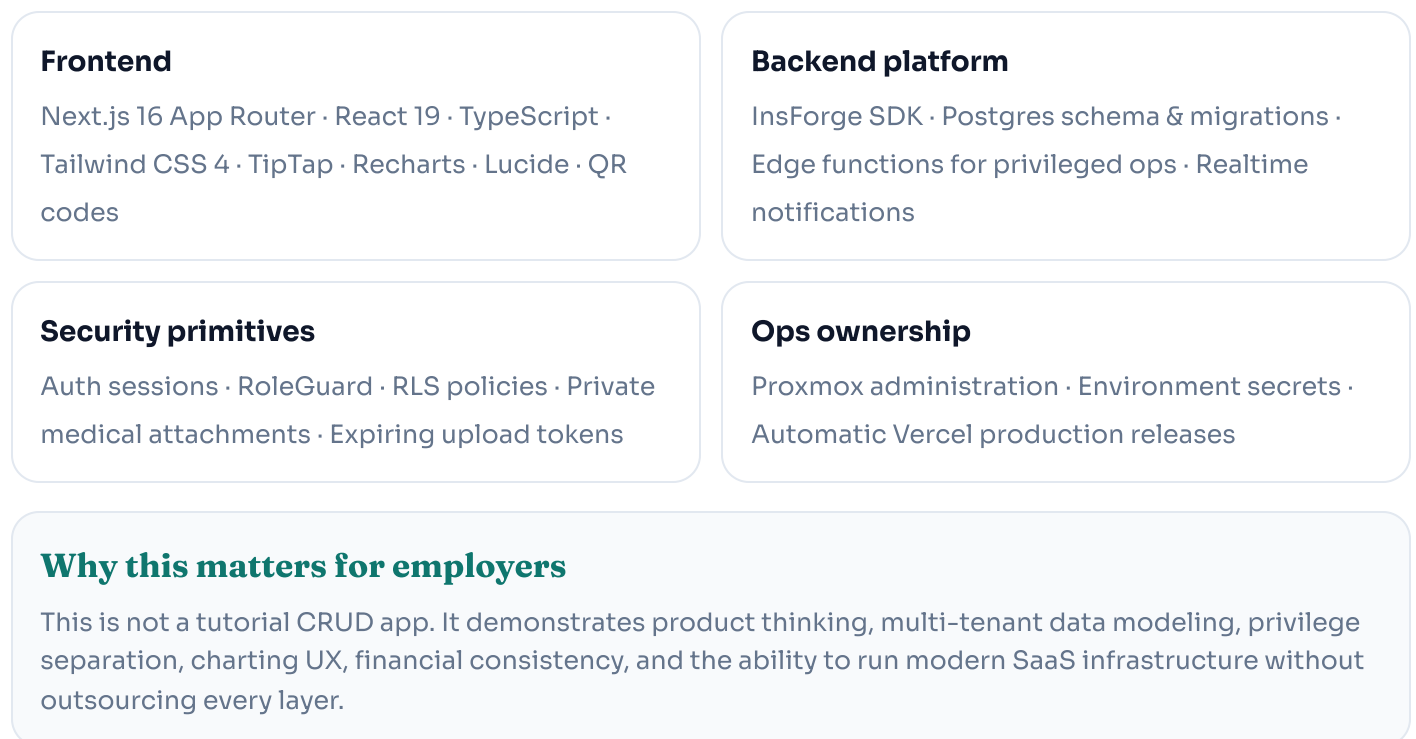
Integrity detail: When a medical-record payment changes, AIDA updates or removes the linked transaction (including heuristic matching for older records), preventing orphaned ledger entries.

Full ownership: app, backend, and infrastructure

The frontend ships on **Vercel** with automatic deployments from version control. The **InsForge** backend (Postgres, Auth, Storage, Edge Functions, Realtime) runs on a **Proxmox** server that I administer myself — virtualization, networking, updates, and uptime.



Authenticated API traffic flows from the Vercel frontend to InsForge services hosted on Proxmox.



Why this matters for employers

This is not a tutorial CRUD app. It demonstrates product thinking, multi-tenant data modeling, privilege separation, charting UX, financial consistency, and the ability to run modern SaaS infrastructure without outsourcing every layer.

HIPAA-aware design and responsible AI use

AIDA handles identifiable dental/health information, so privacy was treated as a product requirement — not an afterthought. This case study claims **HIPAA-aware design** (controls aligned with common HIPAA principles such as minimum necessary access and access control). It does **not** claim formal HIPAA certification, a signed BAA package, or a completed compliance audit.

Implemented safeguards

- Authenticated access for administrative routes
- Role-separated UI (doctor/admin vs assistant)
- Postgres Row Level Security scoped to owners / assistants
- Minimum-necessary assistant patient view
- DB triggers limiting assistant field updates
- Private medical attachment storage (not public hotlinks)
- Time-limited upload tokens managed by service role
- Cascading patient deletion across records & files

Responsible AI in development

- AI used as a supervised coding assistant — not as a clinical decision system
- Human review of generated code, migrations, and UX copy
- TypeScript + lint gates for correctness
- No AI diagnosis / treatment recommendations in the product
- No intentional sharing of real PHI with AI tools for this portfolio
- Final accountability remains with the engineer

Screenshots in this PDF were captured from a live environment and anonymized (names, phones, addresses) before inclusion. Do not treat figures as real patient disclosures.

OUTCOMES

What this project demonstrates

Product craft

Cohesive dental admin UX spanning charts, calendar, odontogram, and finance.

Systems thinking

RLS, roles, linked billing, edge functions, and storage lifecycle designed together.

Ops maturity

Self-managed Proxmox + InsForge backend with automatic Vercel frontend deploys.

A screenshot of a login form for 'AIDA' (Asistente de Información Dental y Administración). The form has a title 'AIDA' and subtitle 'Asistente de Información Dental y Administración'. It contains two input fields: 'Correo electrónico' and 'Contraseña'. Below these is a green button labeled 'Iniciar sesión'. Under the button is a link '¿Olvidaste tu contraseña?' and a radio button. At the bottom is a button with the Google logo and the text 'Continuar con Google'.

Fig. 8 — Authenticated entry point with email/password and Google OAuth.

Let's talk

I'm looking for full-stack / product engineering roles where ownership, privacy-aware systems, and polished user experience matter.

omarboza@gmail.com CV — amazon.github.io/resume

© Omar Boza · AIDA / Dental Family portfolio case study · Built with Next.js, InsForge, Vercel & Proxmox